

Computer Security Exam

Professors M. Carminati & S. Zanero

Milan, 08/07/2020

Last (family) Name _____

First (given) Name _____

Matricola or Codice Persona _____

Have you done any challenges/homework, even partially?	☐ Yes	☐ No
Professor:	☐ Carminati	☐ Zanero
Course:	☐ Milan	☐ Como

Instructions

Instructions:

- Duration: 2 hours.
- The exam is "open book". The students will be allowed to consult only the course material and notes. Be aware that notes and course materials will be accepted only in PDF or digital form, no extra paper or devices (e.g., phones, iPad) is allowed.
- You are not allowed to communicate with other students in any communication form. You will be expelled if, at any time, you do not follow this rule and will not be allowed to take the next session.
- Schemes are good, short answers are recommended.
- Always motivate your answer.
- If your final grade is below 120 (not considering the challenges), you will not be allowed to take the next exam call (even in the next session).
- At the discretion of the professor:
 - Any student may be required to sustain a subsequent oral exam only to confirm the evaluation of the written exams.
 - If you disable the screen sharing or the audio, you will be expelled and not allowed to take the next exam call.

PLEASE, USE THE FORMATTING SPECIFIED IN THE QUESTION

READ CAREFULLY ALL THE POINTS OF EACH QUESTION BEFORE WRITING YOUR ANSWER

Exercise 1 (110 points)

Consider the following program, written in C.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  void encrypt(int backdoor){
6      struct{
7          char tmp[38];
8          char msg[48];
9      } e;
10
11     strcpy(e.msg, "*Decrypted*");
12     printf("Insert the message to encrypt here\n");
13     scanf("%38s", e.tmp);
14     strcat(e.msg, e.tmp); // concatenate the two strings
15     if (backdoor == 1){
16         printf(e.msg);
17     }else{
18         printf("Encrypted: *****\n");
19     }
20 }
21
22 int verify(){
23     struct{
24         char sec1[8];
25         char sec2[16];
26         char *p;
27         int backdoor;
28     } s;
29
30     s.p = s.sec1;
31     s.backdoor = 0;
32     // <-- HERE
33     printf("Please insert the encryption secrets\n");
34     scanf ("%s", s.sec2);
35     scanf ("%7s", s.p);
36     if (strncmp(s.sec2, "FLAG", 4) == 0){
37         encrypt(s.backdoor);
38         return 1;
39     }
40     printf("To be, or not to be, that is the question...\n");
41     abort ();
42 }
43
44 void main (int argc, char **argv){
45     verify();
46 }
```

```
01 #include <stdio.h>
02 #include <stdlib.h>
03 #include <string.h>
04
05 void encrypt(int backdoor){
06     struct{
07         char tmp[38];
08         char msg[48];
09     } e;
10
11     strcpy(e.msg, "*Decrypted*");
12     printf("Insert the message to encrypt here\n");
13     scanf("%38s", e.tmp);
14     strncat(e.msg, e.tmp, 48); // concatenate the two strings
15     if (backdoor == 1){
16         printf(e.msg);
17     }else{
18         printf("Encrypted: *****\n");
19     }
20 }
21
22 int verify(){
23     struct{
24         char sec1[8];
25         char sec2[16];
26         char *p;
27         int backdoor;
28     } s;
29
30     s.p = s.sec1;
31     s.backdoor = 0;
32
33     printf("Please insert the encryption secret\n");
34     scanf ("%s", s.sec2);
35     scanf ("%7s", s.p);
36     if (strcmp(s.sec2, "FL4G", 4) == 0{
37         encrypt(s.backdoor);
38         return 1;
39     }
40     printf("To be, or not to be, that is the question...\n");
41     abort ();
42 }
43
44 void main (int argc, char **argv){
45     verify();
46 }
```

[5 points]

08. The program may be affected by a typical buffer overflow vulnerability. Specify the vulnerable line/s of code. If more than one line is affected, use the following format 00,01,03. Write “no vulnerabilities” in case a class of vulnerabilities is not present.

34

[5 points]

09. Motivate the previous answer.

...

[5 points]

10. The program may be affected by a typical format string vulnerability. Specify the vulnerable line/s of code. If more than one line is affected, use the following format 00,01,03. Write “no vulnerabilities” in case a class of vulnerabilities is not present.

16

[5 points]

11. Motivate the previous answer.

...

[10 points]

12. From now on, assume the program is compiled for the **x86 architecture** (32 bit) and for an environment that adopts the usual **cdecl** calling convention. Furthermore, assume that **no compiler-level or OS-level mitigations** against the exploitation of memory corruption errors is present (unless specified otherwise). Draw the stack layout after the program has executed the instruction at line 32, showing:

- Direction of growth and high-low addresses;
- The name of each allocated variable;
- The content of relevant registers (i.e., EBP, ESP);
- The functions stack frames.

Show also the content of the caller frame.

Use this format:

[H]
[sEBP {main ()}]
[var2 {main ()}]
[var3 (EBP) {main ()}]
[sEBP {fun()}]
[var (ESP) {fun()}]
[L]

[H]
[sEIP {main()}]
[sEBP (EBP) {verify()}]
[backdoor=0 {verify()}]

```
[ *p {verify()}]
[ sec2[12-15] {verify()}]
[ sec2[8-11] {verify()}]
[ sec2[4-7] {verify()}]
[ sec2[0-3] {verify()}]
[ sec1[4-7] {verify()}]
[ sec1[0-3] (ESP) {verify()}]
[L]
```

[20 points]

13. Focus only on the **stack-based buffer overflow** vulnerability. Write an exploit for this vulnerability that **executes the following shellcode**, composed of 7 bytes: **0x31 0xc0 0x40 0x89 0xc3 0xcd 0x80**.

Make sure that you show how the exploit will appear in the process memory with respect to the stack layout right **before and after the execution of the vulnerable line** during the program exploitation. Ensure you describe **all the steps and assumptions** required for a successful exploitation of the vulnerability. Include also any assumption on how you must call the program (e.g., the values for the command-line arguments required to trigger the exploit correctly and/or environment variables, if any).

Use this format:

[before | after]

[H]

[SEBP | not_relevant]

[var2 | not_relevant]

[var3 (EBP) | exploit]

[SEIP | ~address]

[var5 | exploit (ESP)]

[L]

Sec1: not_relevant

Sec2: FLAG+...

```
[H]
[ sEIP {main()}] ESP+4
[ sEBP (EBP) {verify()}] not_relevant
[ backdoor=0 {verify()}] not_relevant
[ *p {verify()}] not_relevant
[ sec2[12-15] {verify()}] 0xc3 0xcd 0x80
[ sec2[8-11] {verify()}] 0x31 0xc0 0x40 0x89
[ sec2[4-7] {verify()}] 0x90 0x90 0x90 0x90
[ sec2[0-3] {verify()}] FLAG
[ sec1[4-7] {verify()}] not_relevant
[ sec1[0-3] (ESP) {verify()}]
```

[L]

[10 points]

14 Focus only on the **stack-based buffer overflow** vulnerability. Assume that the program is compiled only with the correct implementation of the compiler-level mitigation known as “Random Stack Canary” (the address space layout is not randomized, and the stack is executable). Is it effective to prevent **your exploit** at point 13.? If the mitigation technique is effective, explain why and describe how you would modify the buffer overflow exploit to bypass the mitigation. If it is not effective, please explain why.

Yes it is effective + motivation

I would exploit the buffer overflow to:

- Write the FLAG+shellcode on the stack
- Overwrite backdoor=1
- Overwrite *p with the address of EIP

Then I would insert in input at line 35 the address of the shellcode

[10 points]

15. Now focus on the format string vulnerability only. Write an exploit for this vulnerability that executes the previous shellcode you have already allocated in memory. Assume that:

- The address of the return address (*saved EIP*) of the function **verify** is: **0xfffffb5d8** (i.e., where to write)
- The address of the first instruction of the **shellcode** is at: **0xf4d0e349** (i.e., what to write)
- The displacement on the stack of the vulnerable function’s argument is: **2**

Knowing that $dec(0xf4d0) = 62672$ and $dec(0xe349) = 58185$, write the exploit clearly, detailing all the components of the format string and all the steps that lead to a successful exploitation.

To exploit it, we need to write 0xf4d0e349 to 0xfffffb5d8.

As 0xf4d0 > 0xe349, we don't need to swap.

The general format string structure is:

<where to write><where to write + 2>%<low value>c%<pos>\$hn%<high value>c%<pos>\$hn

- Where to write = \xd8\xb5\xff\xff

- Where to write + 2 = \xda\xb5\xff\xff

- we have already written “*Decrypted*.” (12 characters) + 8 characters

- low value = 0xe349 -> 58185 - 12 - 8 = 58165

- high value = 0xf7e3 -> 62672 - 58185 = 4487

Also, the displacement on the stack of the printf’s argument (i.e., the buffer message) is 2, the displacement of “where to write” is 2 + 3 = 5 and the displacement of “where to write + 1” is 6.

Complete exploit: \xd8\xb5\xff\xff\xda\xb5\xff\xff%58165c\$5hn%4507c\$7hn

[5 points]

16. Consider the correct implementation of “Random Stack Canary” mitigation technique. It is effective to prevent **your exploits** at point 15.?

No

[5 points]

17. Consider the previous answer. If the mitigation technique is effective, explain why and describe how you would modify the exploit to bypass the mitigation. If it is not effective, please explain why.

[5 points]

18. Consider the “Address-space layout randomization (ASLR)” mitigation technique. It is effective to prevent **your exploits** at point 15?

Yes the exploit assumes that the address of the buffer is known, at least with a certain degree of precision (assuming that the entropy of the randomisation is enough)

[5 points]

19. Consider the previous answer. If the mitigation technique is effective, explain why and describe how you would modify the exploit (or use a different exploitation technique) to bypass the mitigation. If it is not effective, please explain why.

If ASLR randomizes only the libraries and the stack/heap, using ROP with gadgets in the text section. If the binary is PIE, we can bypass ASLR only if there's another vulnerability which gives a leak of some stack or libc address. In this case, we could use the format string vuln as a leak, but we would have already sent the input to the program --- thus, in this case we have no way to bypass ASLR to exploit the buffer overflow, unless we use more advanced techniques that are out of the scope of this course.

[5 points]

20. Consider the correctly implemented “Address-space layout randomization (ASLR)” mitigation technique. It is effective to prevent your exploits at point 15.??

Yes, same reason as the buffer overflow

[5 points]

21. Consider the previous answer. If the mitigation technique is effective, explain why and describe how you would modify the exploit (or use a different exploitation technique) to bypass the mitigation. If it is not effective, please explain why.

No way, same reason

[5 points]

22. Consider the “Non-executable stack (NX)” mitigation technique. It is effective to prevent **your exploits** at point 13.?

Yes the exploit needs to execute shellcode from the stack which wouldn't be executable.

[5 points]

23. Consider the previous answer. If the mitigation technique is effective, explain why and describe how you would modify the exploit (or use a different exploitation technique) to bypass the mitigation. If it is not effective, please explain why.

Return to libc or ROP.

[5 points]

24. Consider the correctly implemented "Non-executable stack (NX)" mitigation technique. It is effective to prevent your exploits at point 15?

Yes the exploit needs to execute shellcode from the stack which wouldn't be executable.

[5 points]

25. Consider the previous answer. If the mitigation technique is effective, explain why and describe how you would modify the exploit (or use a different exploitation technique) to bypass the mitigation. If it is not effective, please explain why.

Return to libc or ROP.

Exercise 2 (100 points)

“Memento” is a web application where logged in forgetful users write notes for themselves and read them in the future, every time they need it. Consider the following code snippets:

```
1  https://memento.mori/register
2  def register(request):
3      if request.method != "POST":
4          abort(403)
5      q = prepared_stmt("select username from Users where username = :user")
6      result = query(q, [request.username])
7      if result.empty() and request.username.is_alphanumeric():
8          q1 = prepared_stmt('insert into Users (user, role) values (:user,"rookie")')
9          query(q1, [request.username])
10         message = "User " + request.username + " registered successfully!"
11     else:
12         message = "Registration failed"
13     page = "<h1> Registration </h1>"
14     page += message
15     return render(page)
16
17 https://memento.mori/insert_note
18 def insert_note(request):
19     user = check_session(request.cookies['session'])
20     if user is None or request.method != "POST":
21         abort(403)
22     q = prepared_stmt("insert into Notes (user, time, note) values (:user, :time, :note)")
23     query(q, [user.username, time.now(), request.note])
24     return redirect_url("/index")
25
26 https://memento.mori/notes
27 def show_secret_notes():
28     user = check_session(request.cookies['session'])
29     if user is None or request.method != "GET":
30         abort(403)
31     q = prepared_stmt("select note from Notes where user = :user order by time DESCENDING")
32     result = query(q, [user.username])
33     page = page + "<h1> Welcome " + user.username + ". Your secret notes: </h1>"
34     for note in result:
35         page.append(note)
36     return render(page)
```

https://memen.to/register

```
01 def register(request):
02     if request.method != "POST":
03         abort(403)
04
05     q = prepared_stmt("select username from Users where username = :user")
06     result = query(q, [request.username])
07     if result.empty() and request.username.is_alphanumeric():
08         q1 = prepared_stmt('insert into Users (user, role) values (:user,"rookie")')
09         query(q1, [request.username])
10         message = "User " + request.username + " registered successfully!"
11     else:
12         message = "Registration failed"
13     page = "<h1> Registration </h1>"
14     page += message
15     return render(page)
```

https://memen.to/insert_note

```
18 def insert_note(request):
19     user = check_session(request.cookies['session'])
```

```
20     if user is None or request.method != "POST":
21         abort(403)
22     q = prepared_stmt("insert into Notes (user, time, note) values (:user, :time, :note)")
23     query(q, [user.username, time.now(), request.note])
24     return redirect_url("/index")

https://memen.to/notes
27 def show_secret_notes():
28     user = check_session(request.cookies['session'])
29     if user is None or request.method != "GET":
30         abort(403)
31     q = prepared_stmt("select note from Notes where user = :user order by time DESCENDING")
32     result = query(q, [user.username])
33     page = page + "<h1> Welcome " + user.username + ". Your secret notes: </h1>"
34     for note in result:
35         page.append(note)
36     return render(page)
```

Assume that:

- the function `check_session()` securely manages the users' sessions.
- the web application uses a relational DBMS. The structure of the DB can be easily inferred from the queries in the code.

Considering only the information given till here, identify which of the following common classes of web vulnerabilities are for sure present in the “Memento Mori” application.

[5 points]

26. Focus on SQL injection. Is there a vulnerability belonging to this class in the code?

No

[10 points]

27. Motivate the previous answer: If the vulnerability is present, explain: -- (a) The vulnerable lines involved. -- (b) Why it is present. -- (c) How an adversary could exploit it. -- (d) The simplest procedure to remove this vulnerability while preserving the intended functionalities of the page. If the vulnerability is not present, explain why.

All the queries are performed through prepared statements, which work as follow.....

[5 points]

28. Focus on Cross-site scripting (XSS). Is there a vulnerability belonging to this class in the code?

Yes.

[10 points]

29. Motivate the previous answer: If the vulnerability is present, explain: -- (a) The vulnerable lines involved. -- (b) Why it is present. -- (c) How an adversary could exploit it. -- (d) The simplest procedure to remove this vulnerability while preserving the intended functionalities of the page. If the vulnerability is not present, explain why. (please also specify which subclasses of XSS are present, e.g., stored/reflected/DOM-based)

Line: 35, Stores xss

The notes inserted by each user are inserted in the html page without proper escaping/whitelisting: an attacker could insert javascript code in the notes..... An adversary can exploit it by forcing the user to insert a note that contains malicious Javascript code (e.g., <script>...</script>)...

The simplest solution would be to wrap the display of the notes text with htmlentities(), i.e., htmlentities(note).

[5 points]

30. Focus on Cross site request forgery (CSRF). Is there a vulnerability belonging to this class in the code?

Yes

[10 points]

31. Motivate the previous answer: If the vulnerability is present, explain: -- (a) The vulnerable lines involved. -- (b) Why it is present. -- (c) How an adversary could exploit it. -- (d) The simplest procedure to remove this vulnerability while preserving the intended functionalities of the page. If the vulnerability is not present, explain why.

Line: 18 to 24

The user is authenticated only through cookies before a state changing action is performed on the database (the insert). An adversary can exploit it by [...]

The simplest method to fix it it would be to implement CSRF Tokens. Indeed they [...]

[10 points]

32. Write an exploit for one (or more) of the vulnerabilities you have just identified so that an attacker can add a note among the notes of the user “Leonard Shelby”. Please specify all the steps and assumptions you need to make for a successful exploitation, including whether you need to assume interaction with the victim or not.

This exploit leverages the CSRF vulnerability.

The attacker creates a malicious website that when visited performs a POST request to the url https://memento.mori/insert_note with parameter request.note = “Malicious note text here”.

She distributes the link to Leonard Shelby and makes him click on it. The insert will be performed by Leonard and he will be able to see the note among his.

[10 points]

33. Write an exploit for one (or more) of the vulnerabilities you have just identified so that an attacker can make the browser of the user “Hitchhiker” display the string “Don’t panic” (note that this request is different from the one in question 32). Please specify all the steps and assumptions you need to make for a successful exploitation, including whether you need to assume interaction with the victim or not.

This exploit leverages the CSRF vulnerability + the Stored XSS.

The attacker creates a malicious website that when visited performs a POST request to the url https://memento.mori/insert_note with parameter request.note = `<script>alert(“Don’t panic”)</script>`.

She distributes the link to Hitchhiker and makes him click on it. This will perform an insert in the database and when the user Hitchhiker will view his notes his browser will be tricked in executing the javascript code showing the message.

[10 points]

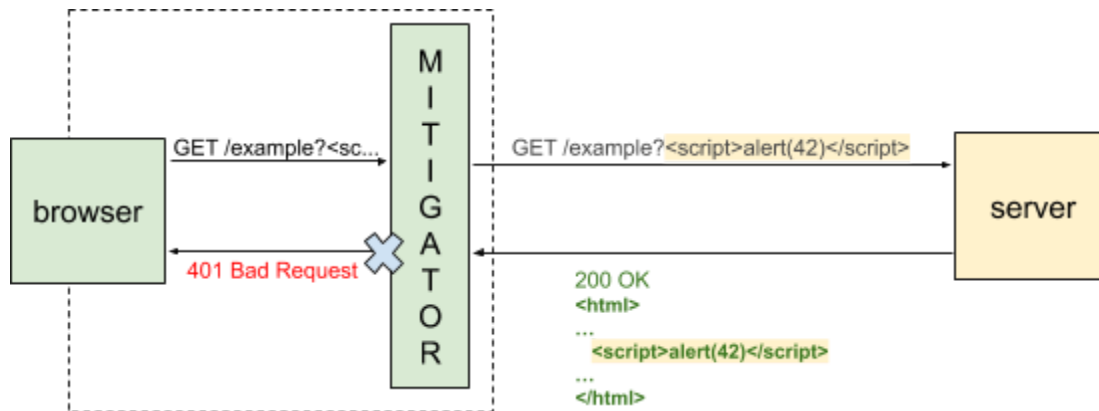
34. Assume that you are a maintainer of “Memento” and that you don’t have direct access to the source code of the app: the only power you have is to add/modify/remove HTTP headers and to set up proxies or anything that won’t change the application code. Please describe clearly the simplest procedure to fix/mitigate the vulnerability/ies you managed to exploit in points 34 and 35. If there are any ways, remember to explain why they work. If there are no ways, state why.

XSS Fix: filtering / CSP set with [...] because [...]

CSRF Fix: CSRF token / Same site cookie because [...]

Consider now the **MITIGATOR**: a mitigator against **XSS vulnerabilities** that can be implemented as a **plug-in of the end-user’s browser**. The plug-in is basically a **proxy that intercepts all the requests and responses made and received by the browser**. The proxy intercepts each request, and **compares the parameters** (for example, the query string) to the **content of the corresponding response**.

If one of the parameters of the request is contained in the HTTP response, and the part of the response where the parameter is contained could be potentially treated as a script by the browser, then the response is blocked before being returned to the browser. The following diagram explains how this mitigation works. In this case, the response is blocked.



[10 points]

35. Please select all the vulnerabilities that this mitigation is effective against

- ☐ Stored XSS
- ☒ Reflected XSS
- ☐ Dom-based XSS

[5 points]

36. Motivate your answer at point 35 w.r.t. Stored XSS. If you deem the mitigation effective against, describe any limitation or edge case where this mitigation could fail or be bypassed.

No: the script will never appear in the HTTP request. There is nothing to match. The Xss is stored in the server and the MITIGATOR can not understand if the code is legit or not.

[5 points]

37. Motivate your answer at point 35 w.r.t. Reflected XSS. If you deem the mitigation effective against, describe any limitation or edge case where this mitigation could fail or be bypassed.

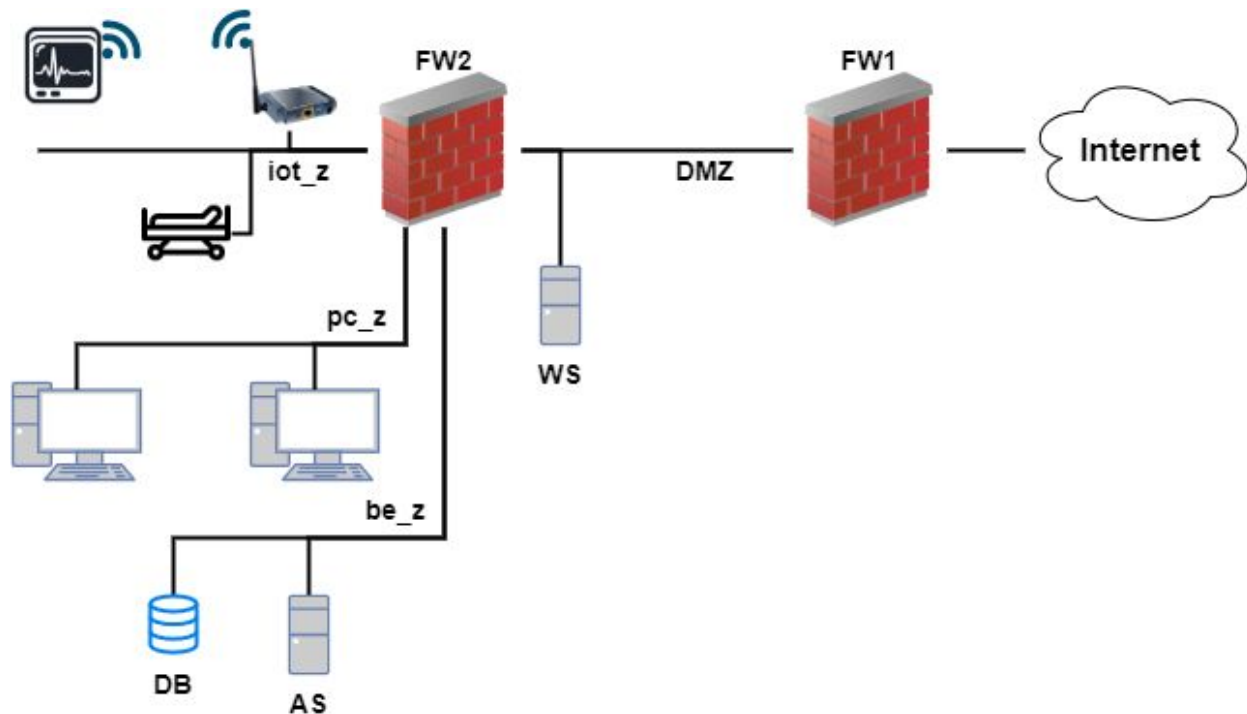
Yes: it is effective as long as the script in the HTTP request is in plain text.
If the url for the malicious site is `https://vulnerable.site/page.html?id=<script>alert(1)</script>` and the page contains `<script>alert(1)</script>` the MITIGATOR will recognize that and block the page from being loaded by the browser. If the parameter of the reflected XSS is somehow processed by the server and changed in the response the MITIGATOR is not able to detect it anymore: for example if the parameter is base64 decoded server-side.

[5 points]

38. Motivate your answer at point 35 w.r.t. Dom-based XSS. If you deem the mitigation effective against, describe any limitation or edge case where this mitigation could fail or be bypassed.

No: in a Dom-based XSS the website returns a page without the XSS, the XSS is then triggered on the user browser because [...]

Exercise 3 (70 points)



Consider the internal network of a technologically advanced hospital. Each medic and a multitude of employees have access to their personal computers connected to PC_Z through a LAN/WLAN. These computers require internet connection (assume HTTP/HTTPs only), and require to access information stored in a database server (DB, on port 10345) through the mediation of the application server (AS on port 9999) .

From the outside (Internet), patients and users have to be able to reach the Web Server, where they can book visits. The web app to book the visits require the web server to be able to access the Application Server, which, in turn, operates on the Database.

Finally, the hospital has a facility-wide wired and wireless network used only to connect medical equipment (IOT_Z), such as x-ray and electrocardiograms machines. These devices need to be reached by the computers of the medics/employees to download data on demand (they accept connections to do so on port 8000).

[15 points]

39. You receive the current firewall (Stateful packet filters firewall) rules for FW1 and FW2 from the hospital staff. Are those rules correct? If not, explain what you would change, add or remove.

- 1) FW1 | ALL:ANY | ANY | ALL:ANY | DENY
- 2) FW1 | ALL:ANY | pc_z -> DMZ | ALL:ALL | ALLOW
- 3) FW2 | ALL:ANY | DMZ -> Internet | ALL:ALL | ALLOW
- 4) FW1 | ALL:ANY | Internet->DMZ | WS_IP:ALL | ALLOW
- 5) FW2 | WS_IP:ANY | DMZ -> be_z | AS_IP:9999 | ALLOW
- 6) FW2 | AS_IP:ANY | be_z -> be_z | DB_IP:10345 | ALLOW
- 7) FW2 | empl+medic IPs:ANY | pc_z -> iot_z | IoTdev_IPs:8000 | ALLOW

ADD FW2 | ALL:ANY | ANY | ALL:ANY | DENY

Change 2) FW2 | empl+medic IPs(buono cmq):ANY | pc_z -> DMZ | ALL:80/443 | ALLOW

Change 3) FW1 | empl+medic IPs:ANY | DMZ -> Internet | ALL:80/443 | ALLOW

Queste 2 sopra valgono come 1, ma se modifico una delle 2 beccano l'altra

ADD FW2 | empl+medic IPs:ANY | pc_z -> be_z | AS_IP:AS_port | ALLOW

Remove 6) FW2 | AS_IP:ANY | be_z -> be_z | DB_IP:DB_port | ALLOW

[20 points]

40. All the hospitals in the country have been required to find some solution to implement security in the IMDs (Implantable Medical Devices) that patients carry around (such as pacemakers). Remember that those IMDs have to be accessible by any medical staff in case of an emergency, but also not accessible by potential attackers. Both cases may be fatal.

The most important question is currently whether to use **symmetric encryption** or a **public-key infrastructure** (PKI) to enable the communication of the IMDs with the hospital equipment. Explain which one you would choose, discussing its strengths and weaknesses.

PKI is better, but good answers will be given points

[10 points]

41. Ambulances may be required to authenticate with IMDs on the field, which would require knowing or retrieving the authentication key while not being inside the local network of the hospital. How would you implement such a feature?

VPN

[10 points]

42. Consider an attacker who has compromised a computer in the zone PC_Z. The attacker wants to mount an attack aimed at intercepting the data exchanged between an X-Ray machine (i.e., a medical device in the zone IOT_Z) and the computer of a specific employee. Assume that the employees' computers have configured a static IP address, while the medical devices use the DHCP protocol to set their network configuration.

- In this scenario, can the attacker use a DHCP poisoning attack to intercept the data exchanged between the X-Ray machine and the computer?
- Would your answer change if the employees' computers used the DHCP protocol instead of a static IP address configuration?

Please describe in detail why it is \ it is not possible to perform the attack, and, in case it is possible, any additional assumptions on the network that you need to perform the attack.

No because they are on two different networks. If also the employees' computers use the DHCP protocol I can mount an attack against the specific computer and redirect the gateway enabling myself to perform a MITM attack.

[15 points]

43. The software running in the X-Ray machine is based on the VeryBrokenRTOS real-time operating system. When doing a TCP three-way handshake, the TCP-IP stack of this operating system chooses the value of the initial sequence number (ISN) as follows:

$$ISN = \text{hash}(\text{source_ip}) \mathbf{xor} \text{hash}(\text{dest_ip}) \mathbf{xor} \text{source_port} \mathbf{xor} \text{dest_port} \mathbf{xor} \text{current_timestamp}()$$

where *hash* is a publicly known and deterministic hash function. Assume that both *hash* and *current_timestamp()* return a 32-bit unsigned integer.

Please describe concisely what is the security issue that this implementation of the TCP-IP stack brings, and describe what attack that it enables, and what are potentially its consequences. Can an attacker positioned in the zone Z_PC mount this attack against the X-Ray machine in the zone Z_IOT? Why?

ISN is not random, they can because the attack works across the network ...

Exercise 4 (50 points)

A new malware just broke out, causing a world-wide infection but no damage until now. First of all, you are able to retrieve and disassemble a sample.

address	binary	disassembled instruction
8049199:	55	push ebp
804919a:	89 e5	mov ebp,esp
804919c:	68 10 0e 00 00	push 0xe10
80491a1:	e8 9a fe ff ff	call 8049040 <sleep@plt>
80491a6:	83 c4 04	add esp,0x4
80491a9:	e8 d8 ff ff ff	call 8049186 <malicious_stuff>
80491ae:	90	nop
80491af:	c9	leave

- 'time()' returns the time as the number of seconds since the Epoch, 1970-01-01 00:00:00 +0000 (UTC).
- sleep() causes the calling thread to sleep either until the number of real-time seconds specified in seconds have elapsed or until a signal arrives which is not ignored.'

Additionally, you execute the malware sample in a widespread dynamic analysis environment that, while the program is executing, collects the list of system calls invoked. The log of system calls provided by the sandbox is as follows:

```
execve("./malware", ["/malware"], 0x7ffd96b406b0 /* 33 vars */) = 0
strace: [ Process PID=372 runs in 32 bit mode. ]
[syscalls during program load, set up of stack/heap/... omitted for the sake of brevity]
clock_nanosleep_time64(CLOCK_REALTIME, 0, {tv_sec=3600, tv_nsec=0}
```

The log of library calls (ltrace) provided by the sandbox is as follows:

```
__libc_start_main(0x80491b1, 1, 0xffb18864, 0x80491c0 <unfinished ...>
sleep(3600
```

[10 points]

44. Looking at the dynamic traces, it seems that the malware is not doing anything malicious. What technique is the malware implementing? How does the technique work in this specific case ?

Dormant_code() ...

[10 points]

45. After a few days from the infection, you were able to retrieve a different sample of the same malware.

address	binary	disassembled instruction
8049199:	55	push ebp
804919a:	89 e5	mov ebp,esp
804919c:	ba 00 0e 00 00	mov edx,0xe00
80491a1:	83 c2 10	add edx,0x10
80491a4:	52	push edx
80491a5:	a1 40 90 04 08	mov eax, 0x8049040 <sleep@plt>
80491aa:	ff d0	call eax
80491ac:	e8 d5 ff ff ff	call 8049186 <malicious_activity>
80491b1:	90	nop
80491b2:	5d	pop ebp
80491b3:	c3	ret

What technique is the malware implementing? How does the technique work in this specific case ?

metamorphism

[20 points]

46. Consider now malware analysis techniques. Briefly discuss which is/are the best techniques to detect the malware implemented before. Motivate your answer.

Static analysis for dormant + dynamic analysis (removing sleep or waiting 1hr) per obfuscation

[10 points]

47. Considering the chosen previous malware analysis technique only, explain which kind of information you would extract in order to develop an effective malware detection system.

signature / heuristics / behaviour